

Invoice System

Apple Store Analytics

Resumen general del proyecto - documentación final

Documentación generada: 2026-05-18 18:53 UTC

1. Visión del proyecto

Sistema de gestión y analítica para un retail tipo Apple Store. Combina datos transaccionales en PostgreSQL (Supabase + Prisma), consultas en lenguaje natural sobre la base de datos, un pipeline RAG sobre manuales operativos en Markdown, y un supervisor multi-agente (LangGraph) con revisión humana (HITL) y generación de cotizaciones en PDF.

2. Stack tecnológico

- Python 3.13+, Prisma Client Python, Supabase (PostgreSQL).
- ChromaDB local (chroma_db_pipeline_completo/) + sentence-transformers + rank-bm25.
- LangChain / LangGraph; LLM vía Groq (supervisor, RAG, ReAct) y Gemini opcional.
- Ragas 0.4.3 (TestsetGenerator) para golden dataset sintético.
- pytest + similitud coseno (embeddings locales) para evaluar respuestas RAG.
- fpdf2 para cotizaciones aprobadas y este documento de resumen.

3. Capa de datos

Esquema principal (Prisma):

- employees - personal de tienda.
- customers - compradores.
- products - catálogo Apple (SKU, categoría, unit_price en USD).
- invoices e invoice_items - ventas y líneas de detalle.

Seeds: productos, empleados, clientes y facturas de muestra (python -m seed.load_seed). CRUD centralizado en crud/prisma_crud.py.

4. Pipeline RAG (rag.main_rag_pipeline_v2)

Corpus por defecto: seed/manual/ (devoluciones, solicitudes-clientes, soporte-tecnico). Flujo: carga de documentos, chunking por párrafos, indexación en Chroma, recuperación avanzada (advanced_rag_query) y síntesis con Groq. API programática: answer_rag_query().

- Índice persistente: chroma_db_pipeline_completo/, colección rag_pipeline_completo.
- Embeddings: paraphrase-multilingual-MiniLM-L12-v2 (mismo modelo en tests golden).

5. Supervisor multi-agente (agents.playground.supervisor)

Grafo LangGraph con clasificación en una sola categoría por mensaje:

- cotizacion - presupuesto con productos y cantidades.
- db_query - ventas, inventario, clientes (ReAct + SQL solo lectura).
- rag_query - políticas y procedimientos según manuales.
- greeting - saludo sin consulta de negocio.

Verificador de confianza (umbral 0.99): si falla, interrupt HITL para corrección humana. Cotización: ReAct sobre products, revisión vendedor (HITL), PDF opcional al aprobar.

CLI interactivo: python -m agents.playground.supervisor

6. Evaluación y calidad RAG

Generación golden (Ragas TestsetGenerator):

- `python -m rag.generate_golden_ragas --per-document --docs-dir seed/manual/`

- Salida: `rag/fixtures/golden/*.jsonl` (~17 preguntas/documento).

Métricas Ragas opcionales en supervisor: `RAG_FAITHFULNESS`, `RAG_CONTEXT_PRECISION`.

`pytest golden (RUN_RAG_GOLDEN=1):` compara `embedding(reference)` vs `embedding(respuesta RAG)`; umbral `RAG_GOLDEN_MIN_COSINE` (default 0.65). Reporte: `rag/fixtures/reports/golden_similarity_pytest_summary.txt`

Última corrida registrada en reporte golden:

- `timestamp_utc`: 2026-05-18T18:24:38.925372+00:00

- `casos_ejecutados_con_metrica`: 57

- `passed`: 44

- `failed`: 13

- `promedio_cosine`: 0.718799

- `umbral_cosine_minimo`: 0.650000

7. Estructura del repositorio (resumen)

- `agents/playground/` - supervisor, `db_react_agent`, `quote_react_agent`, `interactive_chat`.

- `rag/` - ingestion, retrieval, vectorstore, pipelines, `generate_golden_ragas`, `run_ragas_eval`.

- `seed/manual/` - documentación fuente RAG.

- `tests/rag/` - `test_golden_semantic_similarity.py`.

- `diagrams/` - diagramas Mermaid (anexo visual al final de este PDF).

- `pdf/` - generación cotizaciones y este resumen.

8. Comandos esenciales

```
pip install -r requirements.txt && prisma generate
```

```
python connection/prisma_connection_test.py
```

```
python -m seed.load_seed
```

```
python -m rag.main_rag_pipeline_v2
```

```
RUN_RAG_GOLDEN=1 pytest tests/rag/test_golden_semantic_similarity.py -v
```

```
python -m agents.playground.supervisor
```

9. Estado del roadmap

- [Completado] Supabase, Prisma, seeds, RAG + Chroma, supervisor LangGraph, golden Ragas + pytest.

- [Pendiente] Datos sintéticos extensos (30 días) y pipeline JSON / diarios narrativos.

10. Licencia

MIT - ver README.md en la raíz del repositorio.

Anexo A - Diagramas de arquitectura

Las siguientes figuras corresponden a los diagramas Mermaid de la carpeta diagrams/. Se generan automáticamente al construir este PDF (requiere Node.js y npx).

Orden sugerido para presentación: RAG stack (03), supervisor (01), golden pytest (02), ReAct BD/cotización (04).

Figura 1. Supervisor LangGraph - flujo detallado (`agents/playground/supervisor.py`) -
1. Grafo principal (nodos y prioridad de enrutamiento)

Fuente: diagrams/01_supervisor_detallado.md

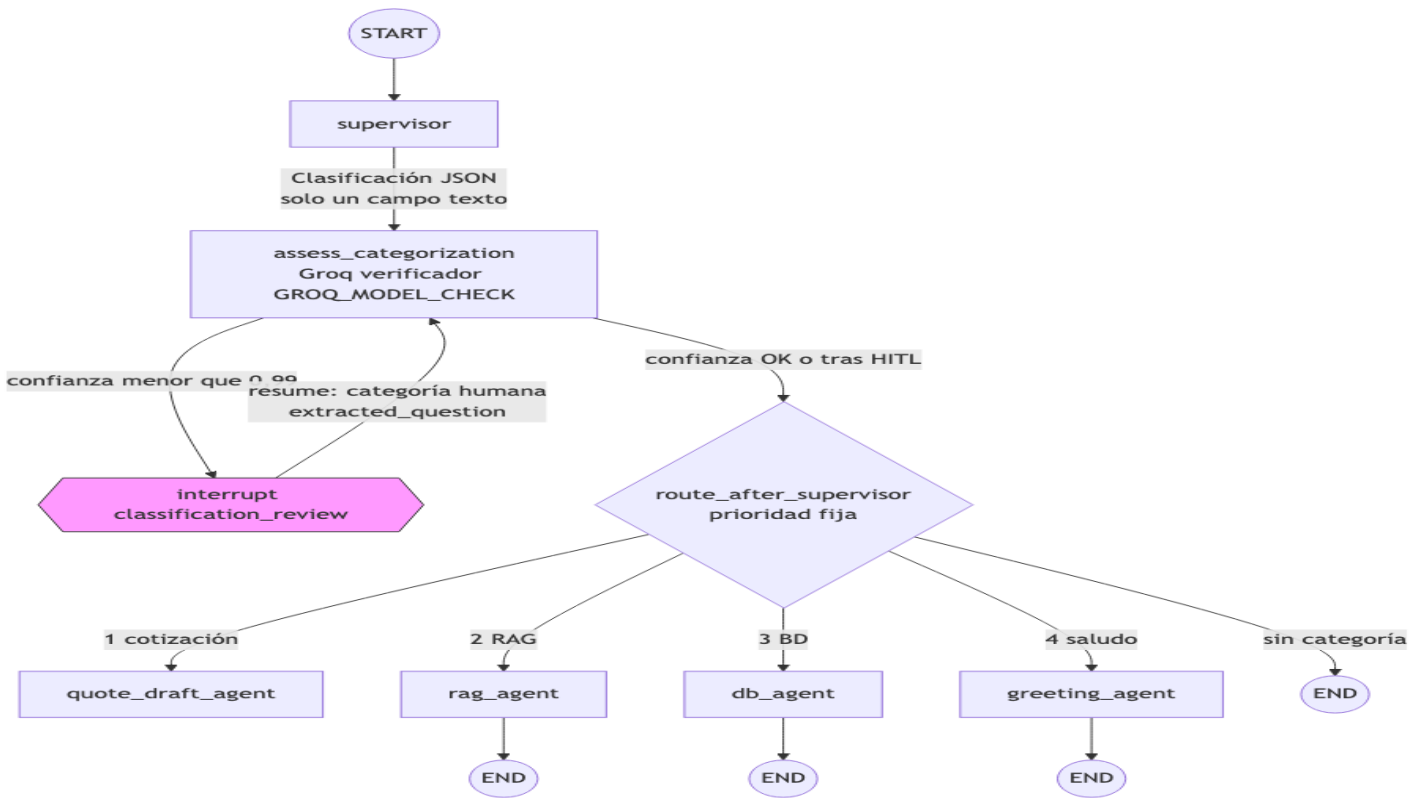


Figura 2. Supervisor LangGraph - flujo detallado (`agents/playground/supervisor.py`) -
2. Ciclo cotización - borrador ? revisión ? PDF opcional

Fuente: diagrams/01_supervisor_detallado.md

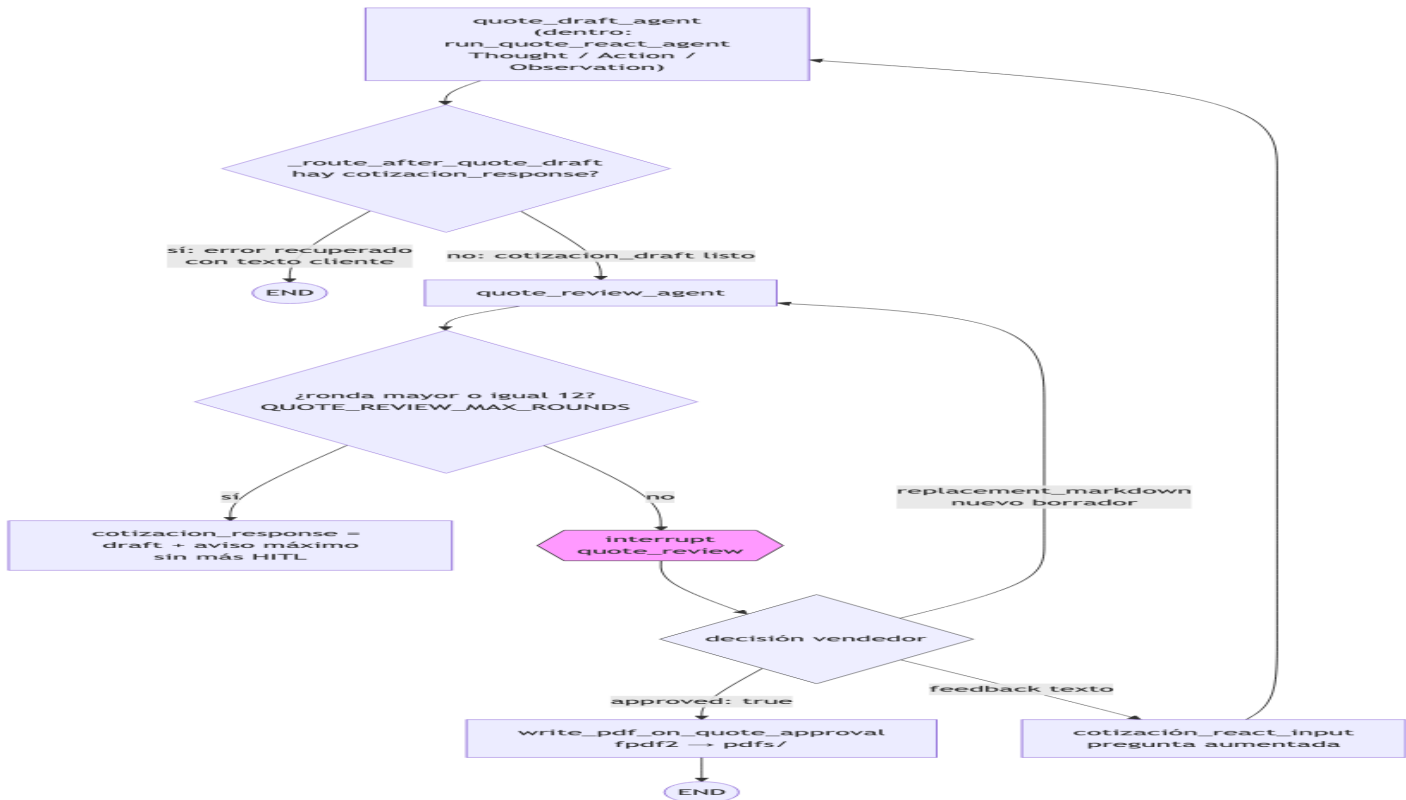


Figura 3. Supervisor LangGraph - flujo detallado (`agents/playground/supervisor.py`) - 3. Qué hace cada worker (referencia rápida)

Fuente: diagrams/01_supervisor_detallado.md

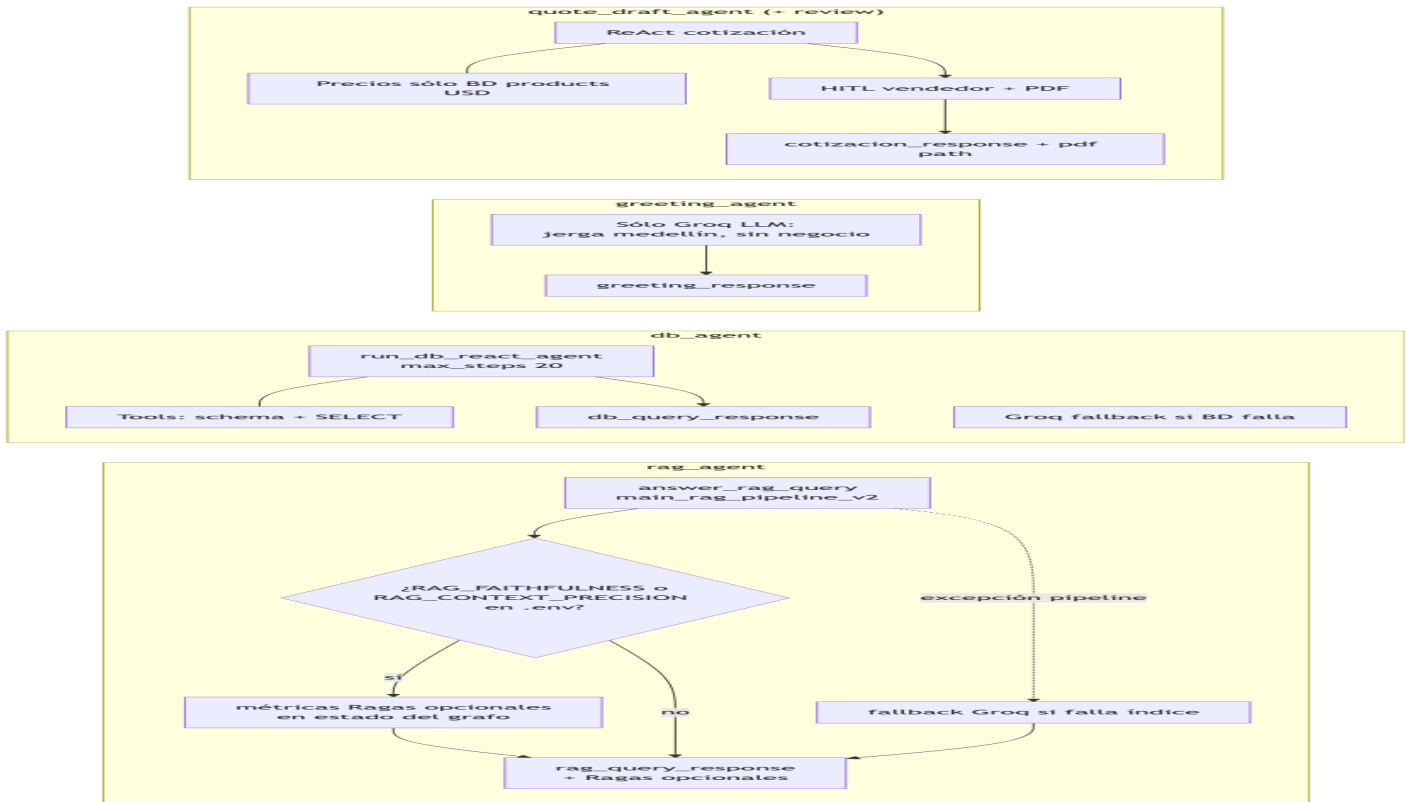


Figura 4. Golden set ? pytest ? similitud semántica - 1. De dónde sale el golden y cómo se evalúa

Fuente: diagrams/02_golden_pytest_similitud.md

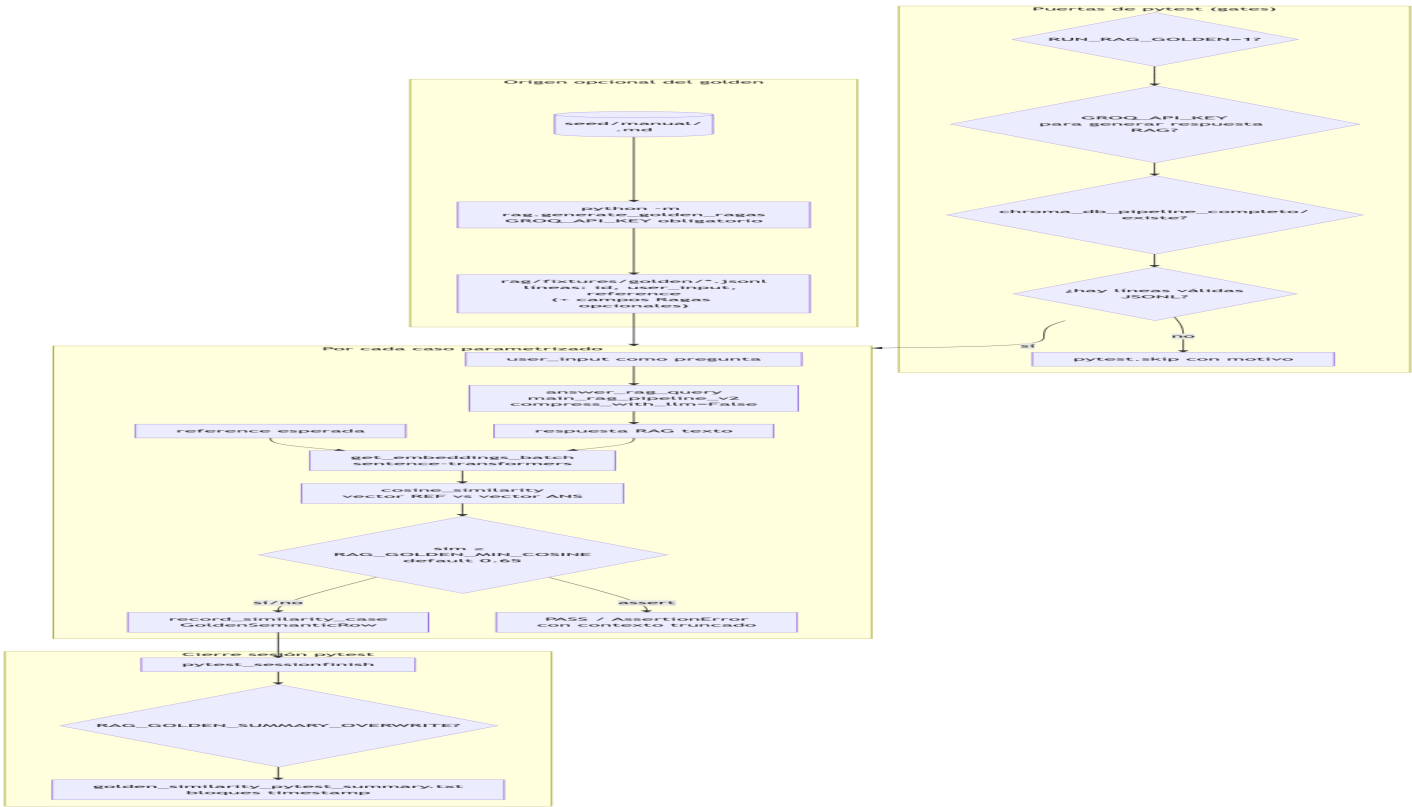


Figura 5. Golden set ? pytest ? similitud semántica - 2. Qué modelo mide la "similitud" (no es el LLM generador del golden)

Fuente: diagrams/02_golden_pytest_similitud.md

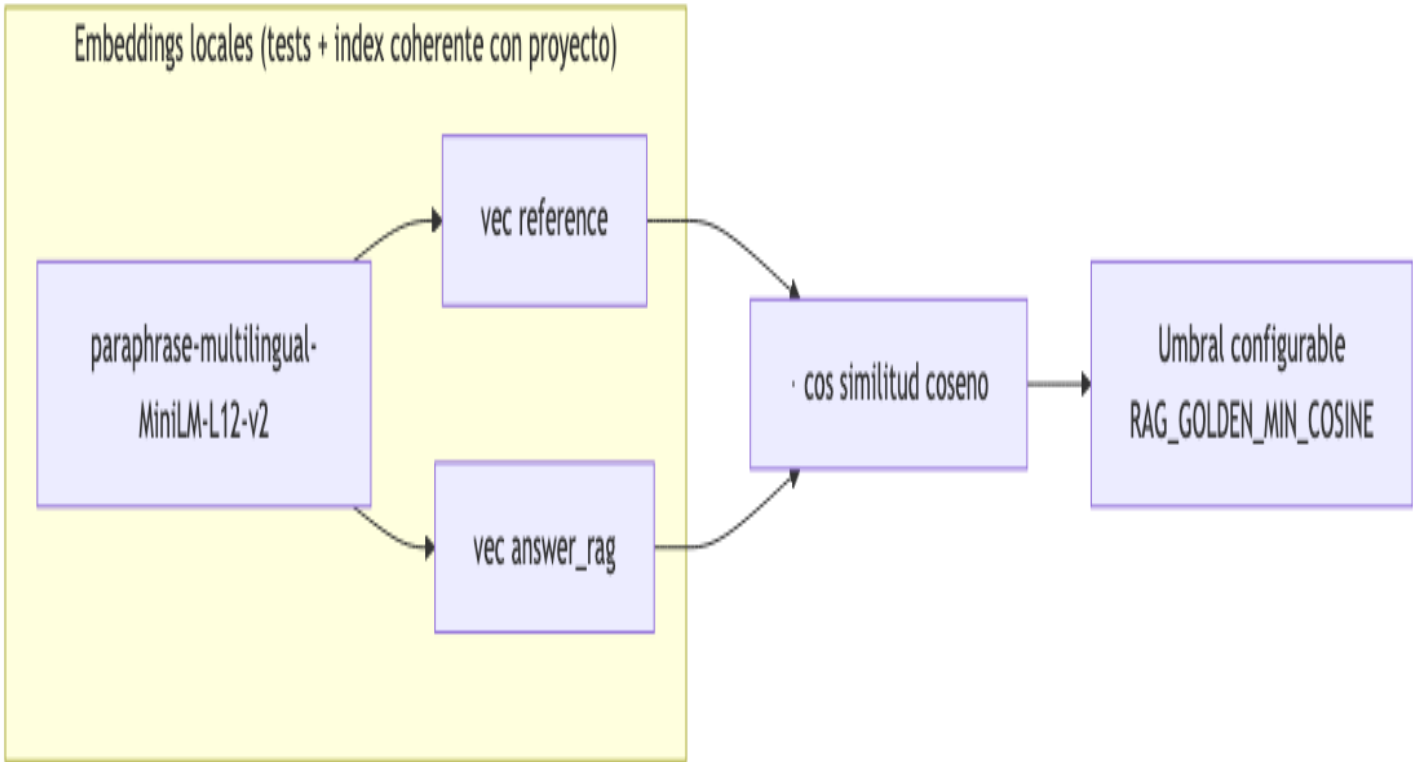


Figura 6. RAG stack - vista para contexto del supervisor (`rag.main\_rag\_pipeline\_v2`)
- Vista general

Fuente: `diagrams/03_rag_stack_inferencia.md`

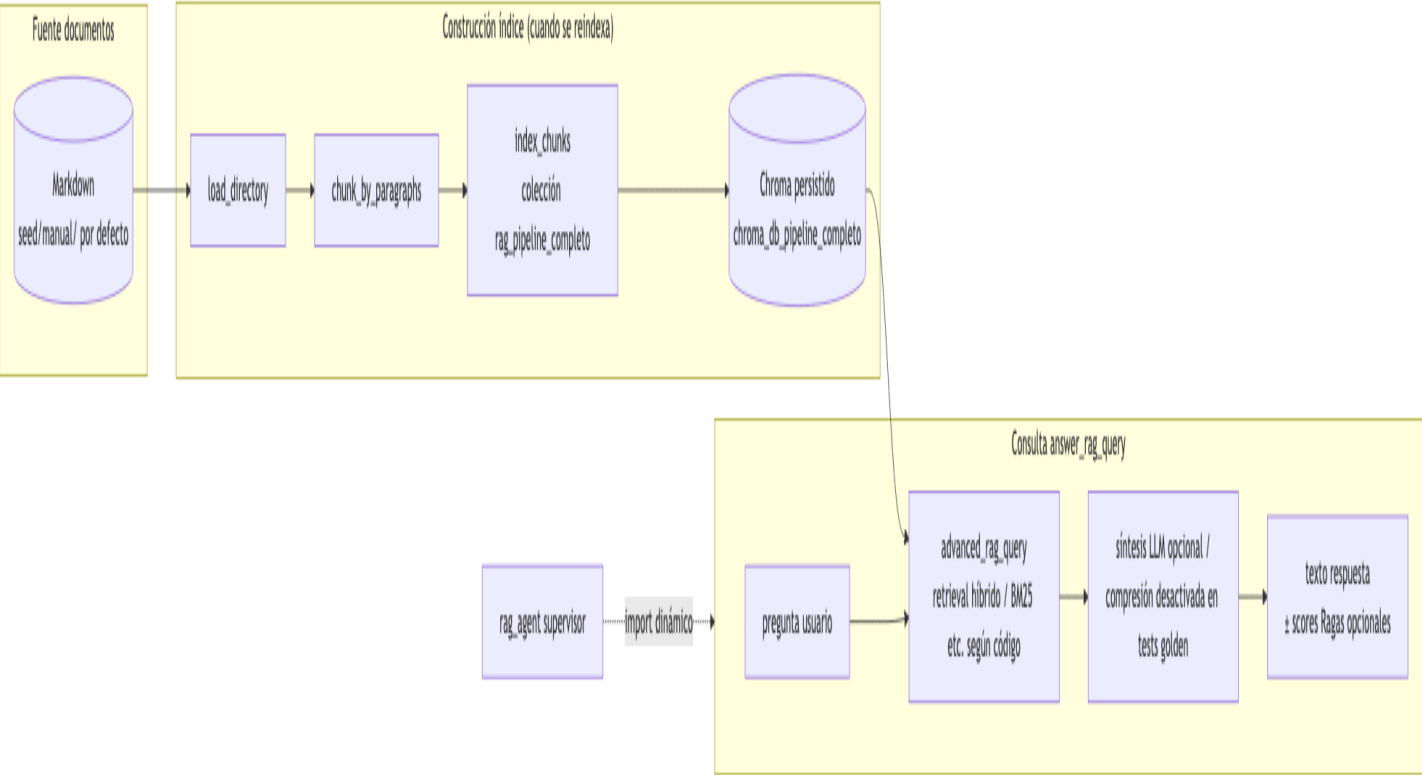


Figura 7. ReAct interno - BD y cotización (`db\_react\_agent` / `quote\_react\_agent`) - Agente BD (solo lectura)

Fuente: diagrams/04_react_bd_cotizacion.md

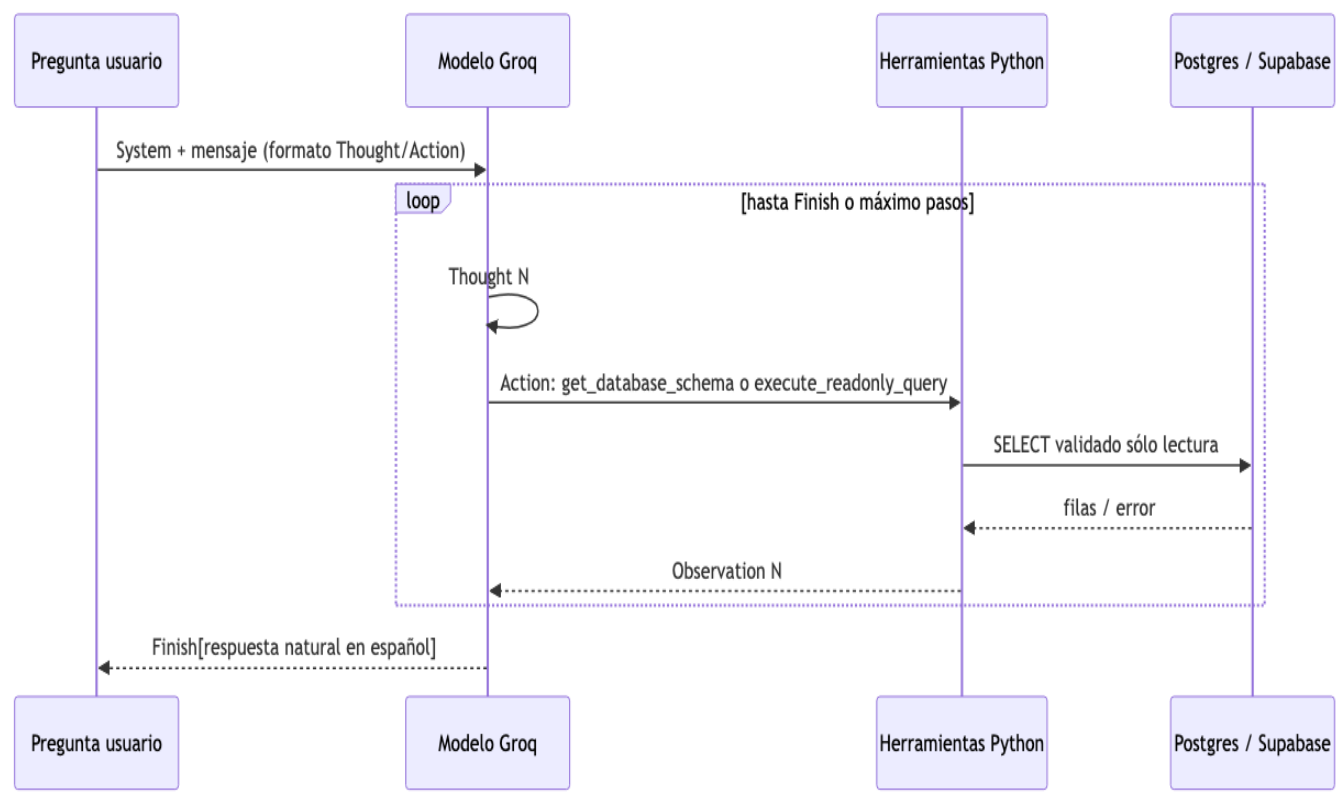


Figura 8. ReAct interno - BD y cotización (`db\_react\_agent` / `quote\_react\_agent`) - Agente cotización (catálogo + documento Markdown)

Fuente: diagrams/04_react_bd_cotizacion.md

